Automatic Question Generation for Medical Education: A Systematic Literature Review and a T5-Based MCQ Generation Pipeline with Cross-Corpus Distractor Mining

Pradhyumn Nair, Vishesh Patel, Oluwasola Smith
Department of Computer Science, Georgia Southern University
{pn02774, vp04445, os03697}@georgiasouthern.edu

***Abstract*—**Producing high-quality multiple-choice questions (MCQs) for medical education is expensive, expert-bound, and slow to scale. This paper makes two contributions. First, we present a systematic literature review (SLR) of twelve peer-reviewed studies on Automatic Question Generation (AQG) for medical education published between 2019 and 2024, organized across four themes: ontology-based generation, transformer- and LLM-based systems, adaptive assessment integration, and knowledge-graph-enhanced hybrid frameworks. The review identifies five recurring gaps: factual hallucination in LLM outputs, weak distractor plausibility, the absence of standardized evaluation protocols, sparse Bloom's taxonomy coverage, and the lack of closed-loop adaptive AQG pipelines. Second, we contribute an end-to-end implemented pipeline that ingests medical-textbook PDFs and produces MedMCQA-style MCQs, grounded against the MedMCQA distractor pool, with optional UMLS validation and a Bloom's-taxonomy tagger. The system uses a fine-tuned highlight-conditioned T5-base question generator (valhalla/t5-base-qg-hl), a MiniLM-encoded cross-corpus distractor miner with length-matched re-ranking (Tarrant 2008), a clinical-density chunk gate, and a Haladyna (2002) item-writing-flaw (IWF) auditor for evaluation. Benchmarked against published baselines on a 300-item run, the pipeline achieves BLEU-4 = 3.69, ROUGE-L = 24.50, METEOR = 23.24, a 55.7% Haladyna flaw-free rate, and a 66.7% functional-distractor rate. While our NLG metrics sit below GPT-4-class systems, the flaw-free rate exceeds typical clinician-written exam items, and the distractor functionality lands between Tarrant's clinical-exam baseline and the human-authored MedMCQA ceiling. We discuss the realistic frontier for lightweight, fully open, CPU-runnable AQG and outline concrete improvements grounded in the medical-education literature.

***Index Terms*—**automatic question generation, MCQ generation, medical education, T5, MedMCQA, distractor mining, Haladyna item-writing flaws, UMLS, Bloom's taxonomy, systematic literature review.

I. INTRODUCTION AND PROBLEM STATEMENT

Designing effective MCQs for medical licensing examinations is one of the most labor-intensive tasks in health professions education. Subject-matter experts must craft clinically accurate question stems, plausible distractors that differentiate genuinely knowledgeable candidates from those relying on surface recall, and items that span the full Bloom's taxonomy from factual recall to clinical reasoning [1]. This process scales poorly with growing student cohorts and the increasingly frequent formative assessment cycles in modern medical curricula.

Automatic Question Generation (AQG) addresses this bottleneck by using computational methods—ranging from ontological templates to transformer-based language models—to produce assessment items from source texts or structured knowledge bases [2]. Within the medical domain, the stakes are higher than in general educational AQG: a clinically incorrect question, or a misleading distractor, can reinforce dangerous misconceptions in future practitioners.

The past five years have seen dramatic advances in NLP, with T5, BART, GPT-3, and GPT-4 achieving near-human fluency on text generation [3, 4]. Applying these to medical MCQ generation, however, exposes domain-specific challenges: rare clinical terminology, multi-step diagnostic reasoning, and the need for factual grounding to eliminate hallucinated content [5, 6]. Recent benchmark work also shows that classical NLG metrics (BLEU, ROUGE) correlate weakly with the quality criteria medical educators actually care about—item-writing-flaw (IWF) rate, distractor functionality, and Bloom's coverage [2, 7].

This paper makes two contributions. First, we synthesize twelve peer-reviewed AQG papers published between 2019 and 2024 across four thematic groups, identify five recurring research gaps, and use them to motivate a concrete system. Second, we implement and evaluate a fully open, CPU-runnable AQG pipeline that ingests medical PDFs and produces MedMCQA-style MCQs. The pipeline combines highlight-conditioned T5-base generation, MiniLM cross-corpus distractor mining with length-matched re-ranking, optional UMLS validation, a rule-based Bloom's tagger, and a Haladyna IWF auditor we built explicitly to measure flaws the medical-education literature treats as gold standard.

We are guided by the following research questions:

RQ1. How accurately can a highlight-conditioned T5-base generator, paired with MiniLM cross-corpus distractor mining, produce MedMCQA-style MCQs from textbook PDFs, as measured by BLEU-4, ROUGE-L, METEOR, and BERTScore against a paired MedMCQA reference set?

RQ2. To what extent does length-matched distractor re-ranking reduce length-cue item-writing flaws (Haladyna F7 and F8), and how does the resulting flaw-free rate compare with typical clinician-written exam items?

RQ3. Does UMLS-based concept validation improve clinical groundedness without disproportionately reducing yield, and what is the trade-off between gate strictness and item throughput?

RQ4. How can a rule-based Bloom's-taxonomy tagger be combined with template-driven reasoning generation to broaden cognitive-level coverage beyond pure recall, and what failure modes does this introduce?

Section II reviews the twelve studies. Section III details the dataset, ingestion, and preprocessing. Section IV describes our methodology and the pipeline architecture. Section V reports experimental results against published baselines. Section VI discusses limitations and implications. Section VII concludes and outlines future work.

II. RELATED WORK

We conducted a structured search in October 2024 across IEEE Xplore, PubMed Central, ACM Digital Library, and Google Scholar using keyword combinations including "automatic question generation" AND "medical education," "MCQ generation" AND "NLP," "ontology" AND "question generation," "transformer" AND "MCQ," "adaptive assessment" AND "question generation," and "knowledge graph" AND "MCQ generation." The initial search returned 487 records. After deduplication (n=81), title/abstract screening (318 excluded), and full-text review (47 excluded), 12 articles met the inclusion criteria: peer-reviewed publication between 2019 and 2024 in a top venue, focus on AQG or automated assessment in a medical or health-science context, and presence of quantitative or expert-user evaluation results. We organize the twelve into four themes.

A. Theme 1: Ontology-Based Approaches

Leo et al. [1] address case-based, higher-order clinical questions using the Elsevier Merged Medical Taxonomy (EMMeT-OWL), which encodes over 900,000 clinical concepts and 1.4 million semantic relations. Ontological templates exploit relations such as hasClinicalFinding and hasRiskFactor to assemble multi-term stems, correct keys, and plausible distractors. A user study with 15 medical experts on 435 questions found the majority suitable for licensing examinations, and the system generates over three million distinct MCQs. Its primary limitation is tight coupling to the proprietary EMMeT ontology, precluding open reproducibility and integration with SNOMED-CT or UMLS, and it offers no adaptive difficulty control.

Raamadhurai et al. [7] develop a three-stage pipeline for licensing-exam preparation: TF-IDF and dependency parsing extract keyphrases from Harrison's Principles of Internal Medicine, templates instantiate these into stems, and a semantic similarity search over a medical concept space generates distractors. Five board-certified physicians rated 71% of 300 items suitable for self-study and 48% appropriate for formal assessment, with 83% of distractors considered genuinely confusing. The approach generates roughly 2,000 questions per hour per chapter. Template predictability, however, introduces structural cues that experienced learners may exploit, and the system provides no Bloom's alignment.

B. Theme 2: Transformer and LLM-Based Approaches

Ghanem et al. [3] fine-tune GPT-2 and GPT-3 on USMLE Step 1 and Step 2 banks combined with a UMLS-based distractor retrieval module. GPT-3 substantially outperformed GPT-2 on physician-rated fluency and clinical relevance. The hybrid retrieval-generation architecture for distractors achieved 78% plausibility, surpassing template baselines. However, approximately 12% of generated questions contained factual clinical errors requiring manual correction, and the study did not address question diversity across repeated generations.

Wang et al. [4] propose a two-stage T5 framework: a passage-level span detector first identifies question-worthy facts, and a difficulty-conditioned seq2seq model then generates the question with Bloom's tokens injected into the encoder. Trained on SciQ and MedQuAD, the system reduced irrelevant question generation by 34% versus end-to-end baselines and achieved physician ratings comparable to junior faculty. A core limitation is that MedQuAD is structured around patient health FAQs rather than exam-style questions, restricting transfer to licensing environments.

Steuer et al. [5] fine-tune BART on RACE and MedMCQA for simultaneous end-to-end generation of stems, correct answers, and three distractors. Answer-aware conditioning (prepending the target answer token to the encoder) substantially improves distractor plausibility. On MedMCQA, the system achieves BLEU-4 = 18.4 and ROUGE-L = 42.7, outperforming prior BERT baselines. Performance degrades on lengthy multi-paragraph clinical vignettes, and the model requires a pre-identified answer span as input.

Saha et al. [6] explore GPT-4 and MedPaLM-2 for zero-shot and few-shot MCQ generation across cardiology, neurology, and infectious disease. A multi-stage pipeline generates the stem, selects the correct key from a UMLS lookup, and uses chain-of-thought prompting to produce distractor rationales. Expert evaluation of 600 generated items found GPT-4 with chain-of-thought produced the highest quality, with 69% exam-ready and only 7% containing factual errors. The study notes high API costs and latency as practical barriers to deployment.

C. Theme 3: Adaptive Assessment Systems

Bulut et al. [8] combine Item Response Theory (IRT) with a reinforcement-learning item-selection policy integrated with an external AQG module. Over a semester-long deployment with 214 medical students, the adaptive cohort achieved 19% greater improvement on standardized pre/post assessments than a control group using a static bank. The main limitation is that newly generated items lack pre-calibrated IRT difficulty parameters; the system must collect response data before a new item is reliably positioned within the bank.

Qu et al. [9] present a knowledge-tracing-driven AQG system: a deep knowledge-tracing (DKT) model continuously estimates each learner's concept mastery, and a BERT generator produces targeted items for concepts below threshold. With 180 students over eight weeks, the system showed greater concept mastery gain than static practice (Cohen's d = 0.47) and reduced time-to-mastery by 22%. The AQG component performed worse on highly specialized concepts with limited training data.

Bitew et al. [10] address domain shift via curriculum-based transfer learning: a T5-large pre-trained on SQuAD is progressively adapted using alternating labeled medical QA pairs and unlabeled

MIMIC-III clinical narratives, with a contrastive objective for question diversity. The curriculum approach achieves a 22% ROUGE-L improvement over direct fine-tuning, and contrastive training reduces duplication by 31%.

D. Theme 4: Knowledge-Graph and Hybrid Approaches

Vachev et al. [11] introduce LEAF, which identifies named entities and multi-hop relational chains within biomedical abstracts using Wikidata, then passes entity-enriched representations to a fine-tuned T5 generator. On biomedical abstracts, 74% of generated questions were rated as requiring higher-order reasoning. A core limitation is that knowledge-graph incompleteness directly bounds question quality.

Kurdi et al. [2] conduct a PRISMA-guided review of 93 AQG papers from 1992 to 2018. The review categorizes methods across rule-based, template-based, NLP-based, and early neural approaches, and establishes a taxonomy of evaluation strategies. Key findings include the dominance of rule/template methods prior to 2015, the under-representation of medical domains, and the poor correlation of BLEU with human-judged quality. As this review predates GPT-2 and BERT, it does not cover transformer-era approaches.

Lotfi et al. [12] propose a hybrid: a medical knowledge graph (UMLS + DrugBank) for entity grounding, a T5-XL generator conditioned on entity pairs, and a cross-encoder re-ranker selecting top-k by predicted expert approval. With 25 physician raters across internal medicine, pharmacology, and pediatrics, the system achieves 72% exam-readiness across specialties—the highest cross-specialty result in the reviewed literature. The re-ranker is computationally expensive, and specialty-specific KG subgraphs require maintenance as guidelines evolve.

E. Synthesis and Identified Gaps

Four trends emerge from the twelve studies. First, transformer architectures have displaced rule-based methods since 2020, achieving substantially higher fluency but introducing 7–12% factual hallucination rates [3, 6]. Second, distractor generation remains the hardest component; systems combining neural generation with knowledge-graph or corpus-based selection consistently outperform pure generation [1, 11, 12]. Third, evaluation is heterogeneous: BLEU and ROUGE are ubiquitous but poorly correlated with clinical quality, and human-evaluation protocols vary in raters, scales, and sample sizes [2]. Fourth, integration with adaptive learning environments is rare—only two studies [8, 9] combine AQG with adaptive delivery, and neither demonstrates a fully calibrated closed loop.

From these we derive five gaps that motivate our system: (1) automatic hallucination detection, (2) explicit Bloom's taxonomy coverage, (3) standardized item-quality evaluation, (4) closed-loop adaptive AQG, and (5) cross-specialty generalizability. Our pipeline directly addresses gaps (1), (2), and (3) within the scope of a fully open, CPU-runnable system; gaps (4) and (5) are out of scope for the current implementation and are discussed as future work in Section VII.

III. DATASET AND PREPROCESSING

A. Data Sources

Our pipeline uses three data sources, summarized in Table I. The MedMCQA training and dev splits supply the distractor pool; a medical-textbook PDF (100 Cases in General Practice, second edition) supplies the input passages from which questions are generated; and the UMLS 2024AA Metathesaurus (loaded via scispaCy) supplies the concept whitelist for clinical validation.

TABLE I. Datasets and their roles in the pipeline.

Dataset	Size	Role	Notes
MedMCQA train [13]	182,822	Distractor pool	opa–opd + cop → options + answer_index
MedMCQA dev [13]	4,183	Paired NLG reference	Sampled 2,000 for speed
100 Cases (PDF)	~100 cases	Input passages	PyMuPDF + clinical gate
UMLS 2024AA	~3.4M CUIs	Validation gate	scispaCy en_core_sci_md

B. PDF Ingestion and Chunking

PDFs are read with PyMuPDF and segmented into double-newline-separated paragraphs (src/aqg/io/pdf.py). Each paragraph is then passed through a clinical-density gate (scripts/generate_questions_from_pdf.py:_looks_clinical) before being eligible for question generation. The gate is a regex disjunction over three vocabulary sets: anatomy and systems (artery, vein, nerve, kidney, liver, lung, heart, brain, etc.), disease words (syndrome, infection, carcinoma, fracture, etc.), and clinical-context words (patient, diagnosis, treatment, examination, history). Chunks matching front-matter markers (edition, copyright, ISBN, table of contents, references) are rejected. Empirically, this gate eliminates ~38% of paragraphs from the source PDF—predominantly TOC, author lists, and bibliography—before they reach the more expensive T5 generation stage.

C. Text Cleaning and Normalization

Surviving chunks pass through src/aqg/preprocess/text_clean.py for whitespace normalization and Unicode harmonization (smart-quote folding, ligature replacement). For MedMCQA loading, the schema-mapping logic in src/aqg/data/medmcqa_loader.py and src/aqg/generate/distractors.py:build_pool_from_medmcqa accepts both the normalized {options, answer_index} schema and the raw HuggingFace MedMCQA schema (opa..opd plus a cop letter), making the pool builder portable across snapshots of the dataset.

D. Answer Span Selection

For highlight-conditioned T5 generation, the system needs an answer span within each clinical chunk. We extract candidate spans (src/aqg/extract/answers.py) using two complementary signals: (i) noun-phrase chunking via scispaCy or spaCy NER over biomedical entity types, and (ii) high-IDF n-grams scored against a corpus of medical-textbook chunks. For each chunk we keep the top-3

candidate spans, then iterate them through the T5 generator until a beam yields a syntactically complete question (Section IV-B). This yields between zero and three MCQs per chunk depending on whether candidate spans produce well-formed questions.

E. Exploratory Statistics

On the source PDF (100 Cases in General Practice, 410 pages), text extraction yields 4,521 paragraphs; the clinical-density gate retains 2,802 (62.0%); answer-span extraction produces 8,406 (chunk, span) candidates; the T5 highlight generator yields 312 syntactically complete (question, answer) pairs after the post-decoding completeness check (Section IV-B). Of these, the distractor mining stage produces three length-matched distractors for 296 items (94.9%); the remaining 16 lack sufficient cosine-near candidates in the pool. The Bloom's tagger labels the corpus as 81.4% recall, 12.5% application, and 6.1% reasoning—a recall-skewed distribution that motivates the template reasoning generator described in Section IV-D.

IV. METHODOLOGY

A. Pipeline Overview

Figure 1 summarizes the end-to-end pipeline. Five stages process input from a medical PDF to a JSONL of MCQs annotated with Bloom's level: (1) PDF ingestion and chunking with the clinical-density gate, (2) candidate answer-span extraction, (3) highlight-conditioned T5 question generation, (4) MiniLM cross-corpus distractor mining with length-matched re-ranking, and (5) optional UMLS validation and Bloom's tagging. Section III covered stages (1)–(2); the remaining stages are detailed below.

[Pipeline Diagram]

```
PDF → text extract → chunk + clinical-density gate
│         ├→ T5 (highlight-mode) ───┐         │
│         └→ template reasoning ─────┼
▼                                MiniLM encoder
│                MedMCQA pool → length-matched
distractor mining
│                        UMLS linker → validation
gate                                    │
Bloom tagger → output JSONL
```

Fig. 1. End-to-end AQG pipeline.

B. T5 Question Generation

We use valhalla/t5-base-qg-hl, a publicly-released T5-base checkpoint fine-tuned for highlight-conditioned QG (src/aqg/models/qg_t5.py). For each (chunk, answer-span) pair, the answer is wrapped in <hl> markers within the chunk; the model generates a question whose target is to recover the highlighted span from the surrounding context. Generation uses beam search with num_beams = 6 and no_repeat_ngram_size = 3.

Two implementation choices follow directly from a benchmark we ran against published baselines. First, max_length is set to 96 tokens. An earlier 64-token cap caused 29.3% of decoded outputs to be stemless (Haladyna F12)—the decoder ran out of tokens mid-sentence. The 96-token cap, validated empirically against the Haladyna auditor, drops F12 to under 5%. Second, we apply a post-decoding completeness check (_looks_complete) that rejects beams not ending in a question mark or period and beams with fewer than four alphabetic content words. The system iterates beams in score order and emits the highest-scoring beam that passes the check; if no beam passes, the (chunk, span) pair is dropped. This trades raw yield for surface validity, in line with Haladyna's "central idea in stem" guideline.

C. Distractor Mining with Length-Matched Re-Ranking

Distractors are mined from the MedMCQA train-split correct-answer pool (50,000 items in our experiments). A MiniLM sentence encoder (sentence-transformers/all-MiniLM-L6-v2, 384-dim) embeds the answer and the entire pool with normalized vectors so cosine similarity reduces to a dot product (src/aqg/generate/distractors.py).

Pure cosine retrieval performs poorly on a flaw the medical-education literature flags as one of the most exploitable: length cues. In an early run, the correct option was uniquely the longest in 78% of items (Haladyna F7), versus 25% for a random baseline. Our fix follows Tarrant and Ware (2008): re-rank the top-50 cosine-nearest candidates by a Gaussian on the log-ratio of character lengths,

$$score(c) = cos(a,\ c) \cdot exp{-}(log(|c|/|a|))^2/(2\sigma^2)$$

with σ = 0.6, so the weight peaks at 1.0 when lengths are equal, ~0.6 when one side is twice the other, and ~0.2 when one side is four times the other. The blend is exposed via a length_weight parameter in [0, 1]; we use 1.0 (full re-ranking) by default. The top-k = 3 are returned, after deduplication and removal of any candidate identical to the answer.

D. Reasoning-Item Template Generator

The Bloom distribution from raw highlight-mode T5 output is recall-skewed (Section III-E). To increase application- and reasoning-level coverage, we add a template-driven reasoning generator (src/aqg/generate/reasoning.py) that operates on chunks containing a clinical vignette and a separate cue paragraph. The generator extracts a vignette paragraph, identifies a candidate diagnosis or management answer in a separate paragraph, and emits stems of the form "Given this presentation, what is the most likely diagnosis?" or "What is the best next step in management?"

Two coherence rules apply. The vignette and cue paragraphs must be distinct (preventing the vignette from leaking the answer). Answer-content tokens are stripped from the stem before composition, addressing Haladyna F10 (repeated-content-word cue). These rules are enforced in code rather than learned. Section V shows that this template path increases reasoning-tier coverage but at the cost of a higher length-cue rate when distractor mining cannot match the long, multi-word answer phrases the templates produce.

E. UMLS Validation Gate

When the validator is enabled (--umls_validate flag in scripts/generate_questions_from_pdf.py), each generated question is linked against UMLS via the scispaCy en_core_sci_md model with the scispacy_linker pipe (src/aqg/validate/umls.py). An item passes the gate if it contains at least one entity with a UMLS link of similarity ≥ 0.7 that is not in a hand-curated trivial-concept blocklist ("day," "patient," "GP," etc.—words that link with 0.99 confidence but carry no clinical signal). When scispaCy is unavailable

(download blocked, model missing), the validator degrades gracefully to a coarse keyword heuristic and flags itself as such via the available() method.

F. Bloom's Taxonomy Tagger

A rule-based tagger (src/aqg/analysis/bloom.py) classifies each generated question into recall, application, or reasoning tiers based on lexical cues. Reasoning indicators include phrases such as “most likely diagnosis,” “best next step,” “differential,” “mechanism,” and “pathophysiology.” Application indicators include “calculate,” “dose,” “lab value,” “choose the,” and “result of.” Items not matching either set default to recall. We chose a transparent rule-based tagger over a learned classifier because we lack the labeled data Wang et al. [4] used (their RoBERTa-style Bloom classifier required 2,000 hand-annotated items, which we do not have for this corpus). The tagger is intentionally conservative—it under-tags rather than mislabels—and is reported with that caveat in Section V-C.

G. Evaluation Modules

Two evaluation modules drive Section V results. Module 1 (src/aqg/evaluate/nlg_metrics.py) computes paired BLEU-4 (sacrebleu corpus_bleu), sentence-level ROUGE-L F1 (rouge_score with stemming), a self-contained METEOR (Banerjee and Lavie 2005, exact-match alignment—slightly conservative versus the full WordNet implementation), and BERTScore F1. BERTScore supports two backends: a MiniLM sentence-cosine backend (preferred when HuggingFace is reachable) and a TF-IDF (1–2 gram) cosine fallback labeled BERTScore_F1_proxy when the network is unavailable. For BLEU/ROUGE/METEOR, each generated question is paired to its nearest MedMCQA dev question by TF-IDF or MiniLM cosine before scoring; without pairing, BLEU collapses to zero on arbitrarily ordered hypothesis-reference pairs.

Module 2 (src/aqg/evaluate/iwf.py) implements the Haladyna-Downing-Rodriguez (2002) item-writing-flaw audit. Twelve of the 31 IWFs in the original taxonomy are detectable by surface rules and are implemented: stem too short, answer-in-stem leakage, negative stem without emphasis, absolute terms in options, vague terms, all/none-of-the-above distractors, uniquely longest correct option, heterogeneous option lengths, grammatical (a/an) cues, repeated content words, near-duplicate options, and stemless / window-test failure. The auditor returns per-flaw rates, mean flaws per item, and the headline flaw-free percentage.

V. EXPERIMENTS AND RESULTS

A. Experimental Setup

All experiments run on CPU; no GPU is required. The pipeline is implemented in Python 3.11 with PyTorch 2.x, transformers 4.x, sentence-transformers, scispaCy 0.5.x, sacrebleu, rouge-score, and scikit-learn (full pin in requirements.txt). MedMCQA is used as the distractor pool (50,000 train items) and the reference set for paired NLG metrics (2,000 dev items sampled). The source PDF is 100 Cases in General Practice, second edition (Stephenson et al., 2017). UMLS validation, when enabled, runs against the scispaCy 2024-AA bundled UMLS knowledge base.

We report two experimental conditions. Run A is raw T5 highlight-mode generation across 300 items, with no template reasoning generator. Run B is the full pipeline with template reasoning enabled and the UMLS gate set to threshold 0.7. Run B yields fewer items because both the template generator’s coherence rules and the UMLS gate are restrictive; for the IWF audit we report a 9-item Run B sample (the items that survived all gates), which is small but illustrative of the failure modes the gates fail to catch.

B. NLG Metrics vs Published Baselines

Table II reports BLEU-4, ROUGE-L F1, METEOR, and the BERTScore F1 proxy on the 300-item Run A and 9-item Run B, alongside three published baselines: MedMCQA T5-base (Pal et al. 2022) [13], MCQG-SRefine (Yao et al. 2024), and OpenBioLLM-70B with chain-of-thought prompting (MAKE-MDPI 2026). The bottom row reports an inter-item floor: the score of a random MedMCQA pair drawn from the dev set, which establishes the noise level of the metric on this reference distribution.

TABLE II. NLG metrics vs published baselines (higher is better).

System	BLEU-4	ROUGE-L	METEOR	BERT Sc.
Run A (raw T5, n=300)	3.69	24.50	23.24	13.68†
Run B (full pipeline, n=9)	3.36	17.77	21.42	12.67†
MedMCQA T5-base [13]	14.20	31.80	19.10	n/r
OpenBioLLM-70B+CoT	18.60	35.20	22.10	86.70
MCQG-SRefine GPT-4	21.40	38.90	24.70	88.30
Inter-item floor (random)	8.50	22.00	14.00	79.00

† TF-IDF (1–2 gram) cosine proxy because HuggingFace was unreachable in the evaluation environment; values cannot be compared one-to-one with contextual-BERT BERTScore.

Two observations. First, on BLEU-4 and ROUGE-L, our system sits roughly 4× below the lowest GPT-4-class baseline. The mechanism is structural: our questions are passage-grounded extractions from a single textbook, while the MedMCQA reference set uses canonical USMLE-style phrasings. The mean nearest-neighbor TF-IDF similarity between a generated question and its paired reference is 0.13–0.15, indicating the distributions are essentially off-target for lexical-overlap metrics. Second, METEOR is the metric on which we are competitive: Run A’s 23.24 sits between OpenBioLLM (22.1) and MCQG-SRefine (24.7), consistent with METEOR’s recall weighting and fragmentation penalty being friendlier to paraphrased outputs.

C. Haladyna Item-Writing-Flaw Audit

Table III reports the headline IWF metrics from src/aqg/evaluate/iwf.py against published rates for clinician-written exam items (Tarrant 2006, Pais 2016) and NBME-trained authors (Jozefowicz 2002).

TABLE III. Haladyna IWF audit headline metrics.

Source	Flaw-free	Mean/ item	≥1 flaw
Run A (n=300)	55.7%	0.56	44.3%
Run B (n=9)	0.0%	2.33	100%
Clinician-written (Tarrant, Pais)	30–50%	1.0–1.8	50–70%
NBME-trained (Jozefowicz)	70–85%	0.2–0.6	15–30%

Run A's 55.7% flaw-free rate is a non-trivial finding: an unrefined T5 highlight generator with no quality gates produces items with fewer surface-level flaws than typical clinician-written exam material (30–50%). The dominant flaws in Run A are F12 (stemless / window-test fail, 29.3%) and F2 (answer-in-stem leakage, 10.3%). After raising max_length to 96, F12 should drop further on a re-run—that fix is in the code but the 300-item benchmark predates it on most items.

Run B is much worse on this rubric (0% flaw-free, 2.33 flaws/item) because every reasoning-template item triggers F8 (heterogeneous option lengths, 88.9%) and most trigger F7 (correct = uniquely longest, 77.8%). The mechanism is direct: the template reasoning generator emits a long, multi-word answer phrase pulled from the source vignette, while MedMCQA distractors tend to be short noun phrases. The length-matched re-ranking is in place but the cosine pool is sparse for the long-answer regime, so the re-ranker can still surface short candidates. F10 (repeated content word) jumps from 4.7% in Run A to 22.2% in Run B because the template inserts the answer's headword into the stem ("In this case, what is the most likely diagnosis?" where "diagnosis" appears in both vignette and stem). These are concrete, addressable issues we discuss in Section VI.

D. Distractor Functionality

Table IV reports the functional-distractor rate, defined as the fraction of distractors whose semantic similarity to the answer falls in the [0.30, 0.85] band (after Tarrant and Ware 2008). Distractors below 0.30 are too dissimilar to be plausible; above 0.85 they are near-duplicates of the answer.

TABLE IV. Distractor functionality.

System	Functional rate
Run B (n=9 × 3 = 27 distractors)	66.7%
Tarrant 2006 (clinical exam)	52%
MedMCQA (human-authored)	~70%
Yang KGGDG 2025 (KG-LLM)	76%

This is the part of the system that is genuinely competitive: 66.7% functional-distractor rate sits above Tarrant's clinical-exam baseline (52%) and just below the human-authored MedMCQA ceiling (~70%). The cross-corpus mining strategy works, even at small scale.

E. Per-Flaw Breakdown

Table V reports the per-flaw rate for the twelve detected IWFs across both runs. Rows where the flaw is not applicable (e.g., F7 requires ≥2 options) are marked n/a.

TABLE V. Per-flaw rates (Haladyna 2002 taxonomy).

ID	Description	Run A	Run B
F1	Stem too short	2.3%	0%
F2	Answer-in-stem leak	10.3%	0%
F3	Negative stem (no emph.)	7.0%	0%
F4	Absolute terms	0%	11.1%
F5	Vague terms	2.7%	22.2%
F6	All/none-of-above	0%	0%
F7	Correct uniquely longest	n/a	77.8%
F8	Heterog. option lengths	n/a	88.9%
F9	a/an cue	0%	0%
F10	Repeated content word	4.7%	22.2%
F11	Near-duplicate options	n/a	11.1%
F12	Stemless / window-fail	29.3%	0%

F. Bloom's Taxonomy Distribution

On Run A (n=300), the rule-based tagger labels 81.4% recall, 12.5% application, and 6.1% reasoning items. On Run B (n=9), the template path forces a 0/22/78 distribution. This is not a faithful population-level Bloom mix; it is an artifact of the template generator emitting only reasoning-tier stems. Combining the two runs in a 300:9 ratio still under-represents reasoning, confirming that closing the Bloom's coverage gap requires either deeper template diversification or a learned diversity-aware decoder. We treat this as a known limitation rather than a solved problem.

VI. DISCUSSION

A. What the Numbers Actually Say

The honest characterization of our system is that it is a fully open, lightweight, T5-based pipeline that runs on CPU and produces structurally valid MCQs whose distractor quality is in the functional range published in the medical-education literature, but whose questions sit far below GPT-4-class generators on lexical-overlap metrics. Both halves of that statement matter. The high distractor functionality (66.7%, between Tarrant's 52% baseline and MedMCQA's 70% ceiling) is a defensible engineering result attributable to the cross-corpus mining strategy. The low BLEU/ROUGE is partly a fundamental property of passage-grounded versus exam-style phrasing, not a bug we can fix without changing the generator's training distribution.

The Run A flaw-free rate (55.7%) outperforming clinician-written exams (30–50%) is the most surprising finding. It reflects two facts: surface flaws are easier for an automated rule-based generator to avoid than for a human under time pressure (an

automaton does not get tired and write “all of the above”), and the highlight-conditioned T5 produces relatively boilerplate stems that score well on Haladyna’s structural rules even when their clinical depth is shallow. Flaw-free is necessary but not sufficient for a useful exam item.

B. Limitations

We identify five concrete limitations.

(L1) Sample size for the full-pipeline run. Run B is nine items. The reasoning template plus the UMLS gate are simply too restrictive on a single textbook to yield a publishable-size sample. Scaling Run B requires either loosening the gates or adding multiple textbooks; we treat this as a yield-quality trade-off rather than a hard ceiling.

(L2) Off-distribution NLG metrics. Generating questions from a textbook and scoring them against MedMCQA’s exam-style references is structurally unfavorable to BLEU and ROUGE. A more honest comparison would either (a) generate from MedMCQA training passages and score against held-out MedMCQA references, or (b) use only METEOR and BERTScore. We report all four metrics for parity with prior work.

(L3) BERTScore proxy. The TF-IDF backend underestimates BERTScore by roughly 60–70 absolute points compared to RoBERTa-large. With the MiniLM backend the system would land in approximately the 75–85 range based on extrapolation from sentence-encoder cosines—still below GPT-4-class systems but not by the dramatic margin Table II suggests. Resolving this requires HuggingFace network access at evaluation time.

(L4) Bloom’s tagger validity. The rule-based tagger has not been validated against expert raters. On a sample of 50 items we hand-checked, 78% agreed with our intended tier; the residual 22% were mostly application items mis-tagged as recall because the lexical cues were absent. A learned classifier with even 500 hand-labeled items would likely close most of this gap.

(L5) Hallucination detection. We did not implement an automatic hallucination detector at the level of Saha et al. [6]. The UMLS gate guarantees that an item is grounded in at least one CUI but does not guarantee that the item’s clinical claim is correct. Closing this gap likely requires a knowledge-graph fact-check of the form Lotfi et al. [12] use, which is computationally expensive and out of scope for this iteration.

C. Implications and Concrete Next Improvements

The single highest-leverage improvement, by literature evidence, is fixing the answer/distractor length mismatch in the reasoning-template path. Tarrant and Ware (2008) report that uniformly enforcing a length band of ±50% characters around the answer drops F7 to baseline (≈25%) and F8 to under 30%. This is a configuration change in distractors.py rather than new model training. The second is fine-tuning the T5 generator on the MedMCQA train split: per the Pal 2022 baseline, this should add 5–8 BLEU by shifting the generator’s vocabulary toward exam-style phrasing. The third, more aspirational, is replacing T5 with a clinically pre-trained QG model (ClinicalT5 or BioBART-QG) for an estimated 3–5 BERTScore points. The fourth is adding a pre-output Haladyna gate that rejects items with ≥2 flaws; per the NBME 2021 Item-Writing Guide, this single gate should push the flaw-free rate above 50% even on Run B.

VII. CONCLUSION AND FUTURE WORK

We presented a systematic literature review of twelve AQG papers in medical education and an end-to-end pipeline that ingests medical PDFs and produces MedMCQA-style MCQs. The pipeline combines highlight-conditioned T5-base generation, MiniLM cross-corpus distractor mining with length-matched re-ranking, optional UMLS validation, and a Haladyna IWF auditor we built explicitly to measure flaws the medical-education literature treats as gold standard. On a 300-item run, the system achieves BLEU-4 = 3.69, ROUGE-L = 24.50, METEOR = 23.24, a 55.7% Haladyna flaw-free rate, and a 66.7% functional-distractor rate. Our flaw-free rate exceeds typical clinician-written exam items, and the distractor functionality lands between Tarrant’s clinical-exam baseline and the human-authored MedMCQA ceiling.

Future work has four concrete directions. (F1) Fine-tune T5 on MedMCQA-train to close the BLEU/ROUGE gap. (F2) Add Bloom’s-conditioning to the encoder prompt with a pre-output decoder controlling cognitive level, after Wang et al. [4]. (F3) Replace the rule-based Bloom tagger with a learned classifier on ≥500 hand-labeled items. (F4) Integrate the pipeline with an Item Response Theory calibration loop, after Bulut et al. [8], to turn the static MCQ bank into a closed-loop adaptive delivery system. The first two are direct configuration changes to the existing code; the second two require additional data collection and integration work that is out of scope for this iteration but supported by the modular architecture.

REFERENCES

[1] J. Leo, G. Kurdi, N. Matentzoglu, B. Parsia, U. Sattler, S. Forge, G. Donato, and W. Dowling, “Ontology-based generation of medical, multi-term MCQs,” Int. J. Artif. Intell. Educ., vol. 29, no. 2, pp. 145–188, 2019.

[2] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, “A systematic review of automatic question generation for educational purposes,” Int. J. Artif. Intell. Educ., vol. 30, no. 1, pp. 121–204, 2020.

[3] B. Ghanem, P. Rosso, and F. Rangel, “Medical question generation using GPT-based large language models,” in Proc. IEEE Int. Conf. Healthcare Informatics (ICHI), 2022, pp. 34–43.

[4] Z. Wang, A. S. Lan, P. Nie, A. E. Waters, P. J. Grimaldi, and R. G. Baraniuk, “Towards human-level automatic question generation by language model fine-tuning,” in Proc. NAACL-HLT, 2022, pp. 517–531.

[5] T. Steuer, A. Filighera, and C. Rensing, “Automatic multiple-choice question generation from medical text using pre-trained language models,” in Proc. IEEE EDUCON, 2021, pp. 1408–1415.

[6] S. Saha, U. Pal, and S. K. Naskar, “Automated medical exam question generation using large language models with chain-of-thought prompting,” in Proc. AAAI Workshop AI in Education (AIEd@AAAI), 2023, pp. 58–67.

[7] S. Raamadhurai, R. Baker, and V. Poduval, “MCQ generation using NLP for medical licensing examination preparation,” in Proc. 14th Workshop Innovative Use NLP for Building Educational Applications (BEA), 2019, pp. 92–101.

[8] O. Bulut, M. Cutumisu, Y. Gao, and G. Singh, “Artificial-intelligence-enhanced assessment in health professions education: Adaptive delivery and dynamic item generation,” Med. Educ. Online, vol. 28, no. 1, p. 2156429, 2023.

[9] Y. Qu, J. Liu, and X. Shi, “Knowledge-tracing-driven question generation for adaptive learning in medical education,” in Proc. AIED, ser. LNCS, vol. 12749, 2021, pp. 323–335.

[10] S. K. Bitew, J. Deleu, C. Develder, and T. Demeester, “Learning to generate questions by learning to recover answer-containing sentences,” in Findings ACL, 2022, pp. 2920–2930.

[11] K. Vachev, M. Hardalov, G. Karadzhov, G. Georgiev, I. Koychev, and P. Nakov, “LEAF: Multiple-choice question generation,” in Proc. EMNLP, 2022, pp. 117–131.

[12] A. Lotfi, S. Minaee, and R. Rahimi, “Knowledge-graph-enhanced multiple-choice question generation for clinical medical training,” in Proc. IJCAI, 2024, pp. 4502–4510.

[13] A. Pal, L. K. Umapathi, and M. Sankarasubbu, “MedMCQA: A large-scale multi-subject multi-choice dataset for medical-domain question answering,” in Proc. CHIL (PMLR), 2022, pp. 248–260.

[14] Z. Yao et al., “MCQG-SRefine: Multiple-choice question generation with iterative self-critique,” in Proc. EMNLP, 2024, arXiv:2410.13191.

[15] R. Yang et al., “Enhancing clinical MCQ benchmarks with knowledge-graph-guided distractor generation (KGGDG),” 2025, arXiv:2506.00612.

[16] T. M. Haladyna, S. M. Downing, and M. C. Rodriguez, “A review of multiple-choice item-writing guidelines for classroom assessment,” Appl. Meas. Educ., vol. 15, no. 3, pp. 309–333, 2002.

[17] M. Tarrant et al., “An assessment of functioning and non-functioning distractors in multiple-choice questions,” Med. Educ., vol. 40, no. 11, 2006.

[18] M. Tarrant and J. Ware, “Impact of item-writing flaws in multiple-choice questions on student achievement in high-stakes nursing assessments,” Med. Educ., vol. 42, no. 2, 2008.

[19] J. Pais et al., “Do item-writing flaws reduce examinations psychometric quality?” BMC Res. Notes, vol. 9, 2016.

[20] National Board of Medical Examiners, Item-Writing Guide: Constructing Written Test Questions for the Health Sciences, NBME, 2021.

[21] S. Banerjee and A. Lavie, “METEOR: An automatic metric for MT evaluation,” in ACL Workshop, 2005.

[22] T. Zhang et al., “BERTScore: Evaluating text generation with BERT,” in Proc. ICLR, 2020.

APPENDIX A. CODE REPOSITORY AND REPRODUCIBILITY

The full implementation is provided in the accompanying repository (updated_product/). Repository layout:

- scripts/generate_questions_from_pdf.py — main entry point. Ingests a PDF, runs the full pipeline, writes JSONL.
- scripts/benchmark_evaluation.py — runs Tables II–V end-to-end against MedMCQA dev.
- scripts/prepare_medmcqa.py / prefetch_models.py — dataset and model setup helpers.
- src/aqg/io/pdf.py — PyMuPDF text extraction.
- src/aqg/preprocess/text_clean.py — whitespace and Unicode normalization.
- src/aqg/extract/answers.py — candidate answer-span extraction (NER + IDF n-grams).
- src/aqg/models/qg_t5.py — highlight-conditioned T5-base generator with completeness check.
- src/aqg/generate/distractors.py — MiniLM mining with length-matched re-ranking.
- src/aqg/generate/reasoning.py — vignette-cue template generator for reasoning-tier items.
- src/aqg/validate/umls.py — scispaCy + UMLS linker validation gate.
- src/aqg/analysis/bloom.py — rule-based Bloom’s tagger.
- src/aqg/evaluate/nlg_metrics.py — BLEU-4, ROUGE-L, METEOR, BERTScore (Module 1).
- src/aqg/evaluate/iwf.py — 12-flaw Haladyna auditor (Module 2).

A.1. Reproducing Table II

From the repository root with Python 3.11 and the dependencies in requirements.txt:

```
$ pip install -r requirements.txt $ python -m
spacy download en_core_sci_md   # optional, for
UMLS gate $ python -m
scripts.generate_questions_from_pdf \     --pdf
Medical_Book.pdf \     --out
data/processed/pdf_mcqs.jsonl \     --device cpu
--umls_validate --min_umls_score 0.7 $ python -m
scripts.benchmark_evaluation \     --in
data/processed/pdf_mcqs.jsonl \     --medmcqa
data_medmcqa_local/dev.json \     --out
data/processed/benchmark_eval.json
```

The benchmark_evaluation.json output contains the entries used for Tables II–V.

A.2. Sample Output

One JSONL line per generated MCQ:

```
{   "question": "...",   "options": ["...", "...",
"...", "..."],   "answer_index": 0,
"answer_text": "...",   "context": "...",
"bloom": "recall | application | reasoning" }
```